

二叉树 + 二叉搜索树 (BST)

1. 树的术语

术语	定义
Degree(节点的度)	该节点的直接孩子数
Degree of Tree	所有节点度的最大值
Parent/Child/Sibling	父/子/兄弟
Leaf/Terminal	度为 0 的节点 (没有孩子)
Path	节点序列 $n_1, n_2, \dots, n_k$
Path Length	路径上的 边数 (不是节点数)
Depth	从根向下的边数, 根 = 0
Height	到最深叶子的边数, 叶子 = 0
Ancestor	从节点到根的路径上所有节点
Descendant	该节点子树中所有节点

考点

深度 $\neq$ 高度: 深度从上往下, 高度从下往上。
叶子 $\neq$ 孩子: 叶子是度 = 0 的节点, 孩子是有父节点的节点。
路径长度 = 边数 (不是节点数)

3. 四大遍历 (考试必默写)

```
// 先序: 根→左→右
void preorder(TreeNode* r) {
    if(!r) return;
    printf("%d\n", r->data);
    preorder(r->left);
    preorder(r->right);
}

// 中序: 左→根→右
void inorder(TreeNode* r) {
    if(!r) return;
    inorder(r->left);
    printf("%d\n", r->data);
    inorder(r->right);
}

// 后序: 左→右→根
void postorder(TreeNode* r) {
    if(!r) return;
    postorder(r->left);
    postorder(r->right);
    printf("%d\n", r->data);
}
```

关键: 三种递归只差 printf 的位置。

2. 二叉树三性质

- 1. 第 i 层最多 2^{i-1} 个节点 ($i \geq 1$)
 - 2. 深度 k 的树最多 $2^k - 1$ 个节点
 - 3. $n_0 = n_2 + 1$ (叶子数 = 度 2 节点数 + 1)
- 性质 3 证明: $n = n_0 + n_1 + n_2, B = n_1 + 2n_2, n = B + 1$ 。
代入得证。
空链接数: n 个节点的二叉树有 $n + 1$ 个 NULL 链接。

二叉树类型

斜二叉树: 全左或全右 $\rightarrow$ 最坏深度 $O(N)$ 。
完全二叉树: 除最底层, 其他全满。最底层从左到右连续。

表达式树

叶子 = 操作数, 内部节点 = 运算符。
后缀表达式 $\rightarrow$ 表达式树: 用栈构建。读到操作数 push, 读到运算符 pop 两个当孩子再 push。
三种遍历对应: 中序 $\rightarrow$ 中缀, 后序 $\rightarrow$ 后缀, 先序 $\rightarrow$ 前缀。

中序遍历—迭代版

```
void iterInorder(TreeNode* r) {
    Stack S; init(&S);
    for(;;){ // 也可 while(t&&!isEmpty(&S))
        for(;; r = r->left) push(&S, r);
        if(isEmpty(&S)) break;
        r = pop(&S); visit(r);
        r = r->right;
    }
}
```

层序遍历—用队列

```
void levelorder(TreeNode* r) {
    if(!r) return; Queue q; init(&q);
    enqueue(&q, r);
    while(!isEmpty(&q)){
        TreeNode* t = dequeue(&q);
        visit(t);
        if(t->left) enqueue(&q, t->left);
        if(t->right) enqueue(&q, t->right);
    }
}
```

遍历结果速记

遍历	顺序	用途
先序 (Pre)	根 → 左 → 右	目录列表
后序 (Post)	左 → 右 → 根	目录大小、表达式求值
中序 (In)	左 → 根 → 右	BST 输出有序序列
层序	BFS	逐层遍历

4. 线索二叉树

动机：n 个节点的二叉树有 n + 1 个 NULL。用来指向中序前驱/后继。

规则：

- 1. Tree 上->Left=NULL → 指向中序前驱
- 2. Tree 上->Right=NULL → 指向中序后继
- 3. 加头节点，无悬空线索

节点结构：data, left, right, leftThread, rightThread

5. 二叉搜索树 (BST)

定义：左子树所有值 < 根 < 右子树所有值。键值互异（或可处理重复）。

Find — O(h)

```
TreeNode* find(TreeNode* r, int x) {
    if(!r) return NULL;
    if(x<r->data) return find(r->left,x);
    if(x>r->data) return find(r->right,x);
    return r;
}
// 迭代版: while(r){if(x<r->data)r=r->left;else if...else return r;}
```

FindMin / FindMax — 一直左 / 一直右

```
TreeNode* findMin(TreeNode* r) {
    if(!r)return NULL;
    if(!r->left)return r;
    return findMin(r->left);
}
TreeNode* findMax(TreeNode* r) {
    if(!r)return NULL;
    while(r->right) r=r->right;
    return r;
}
```

Insert — 在 NULL 处创建新节点

```
TreeNode* insert(TreeNode* r, int x) {
    if(!r) {
        TreeNode* n=(TreeNode*)malloc(sizeof(TreeNode));
        n->data=x; n->left=n->right=NULL;
    }
```

```
        return n;
    }
    if(x<r->data) r->left=insert(r->left,x);
    else if(x>r->data) r->right=insert(r->right,x);
    return r;
}
```

易错点

r 为 NULL 时直接 r->data=x → 段错误。必须先 malloc。忘了 root->left=insert(...) 来接返回值。

Delete — 三情况

```
TreeNode* delete(TreeNode* r, int x) {
    if(!r)return NULL;
    if(x<r->data) r->left=delete(r->left,x);
    else if(x>r->data) r->right=delete(r->right,x);
    else { // 找到了!
        // Case 1+2: 0或1个孩子
        if(!r->left){TreeNode*t=r->right;free(r);return t;}
        if(!r->right){TreeNode*t=r->left;free(r);return t;}
        // Case 3: 两个孩子 → 找替身
        TreeNode* min=findMin(r->right); // 右子树最小
        r->data=min->data;
        r->right=delete(r->right,min->data); // 递归删替身
    }
    return r;
}
```

考点

Case 3 核心：找左子树最大或右子树最小替换值，然后删替身（替身必是 Case 1 或 2）。
不要找整棵树的最大！ 应该找左子树的最大。

易错点

忘了 free。
Case 2 的 left/right 写反（只有一个左孩子却返回了右孩子）。
findMax(r) 找整棵树最大值 → 应 findMax(r->left)。
delete(max,max->data) 参数错误 → 应 r->left=delete(r->left,max->data)。

平均情况分析

BST 的深度取决于插入顺序。
随机插入 → 平均深度 O(log N)；升序/降序插入 → 深度 O(N)（退化为链表）。

懒惰删除

加 flag 标记删除，不真正 free。
好处：不用 malloc 就能重插入。坏处：dead node 太多影响效率。

二叉树操作复杂度

操作	时间复杂度	备注
Find / FindMin / FindMax	$O(h)$	h= 树高
Insert	$O(h)$	在 NULL 处创建
Delete	$O(h)$	Case 3 转 Case 1/2
四种递归遍历	$O(N)$	每个节点访问一次
层序遍历	$O(N)$	用队列
BST 随机插入平均深度	$O(\log N)$	
BST 最坏深度	$O(N)$	有序插入